Comparison of Apriori Algorithm for Frequent Itemset Mining with State-of-the-Art Algorithms and Optimization Strategies

Syed Hamza Mukhtar Bukhari, Muhammad Moosa, and Muhammad Saim
Dept. of Artificial Intelligence, Faculty of Computer Science and Engineering
GIK Institute of Engineering Sciences and Technology, Topi, Pakistan
bukhari.hamzamukhtar@gmail.com | msaim22112003@gmail.com | actuallymoosa@gmail.com

***Abstract*—Frequent itemset mining (FIM) is foundational to association rule generation yet remains computationally intensive at scale. This paper presents a rigorous empirical comparison of the classical Apriori algorithm against SHFIM-dECLAT (Al-Bana et al., 2022), a modern vertical diffset-based mining algorithm, evaluated across three standard benchmark datasets — Chess, Connect, and Accidents — under four minimum support thresholds ranging from 80% to 95%. Two Apriori optimization strategies are proposed and empirically evaluated: a bitmap-based transaction encoding and a TID-list intersection approach. SHFIM-dECLAT achieves up to 233× runtime reduction over Apriori on the Chess dataset at 80% minimum support, and the TID-list optimization delivers up to 10.7× speedup over baseline Apriori on Connect at 95% support. The bitmap optimization consistently underperformed the baseline, attributed to Python interpreter overhead negating the theoretical bitwise operation gain. Results confirm that vertical data format algorithms offer structurally superior performance on dense datasets, while also revealing that algorithmic correctness in distributed implementations does not automatically transfer to single-node Python ports — a limitation disclosed in full.**

***Keywords*—frequent itemset mining; Apriori; ECLAT; diffsets; vertical data format; TID-list; bitmap optimization; benchmark datasets**

I. INTRODUCTION

Frequent Itemset Mining (FIM) stands as a foundational pillar within the domain of data mining, tasked with uncovering sets of items that consistently co-occur across massive transactional databases. The extraction of these patterns is critical for generating actionable association rules, which drive decision-making processes across a multitude of industries. In retail analytics, FIM powers market basket analysis to uncover product bundling opportunities. In healthcare informatics, it facilitates the discovery of co-occurring symptoms or medication patterns within clinical records. Furthermore, FIM is instrumental in financial sectors for fraud detection by isolating suspicious, recurring transactional behaviors [1].

Despite its immense utility, FIM remains computationally demanding. The classical Apriori algorithm [1], while conceptually elegant, relies on a breadth-first search and a generate-and-test paradigm that introduces severe performance bottlenecks. Primarily, Apriori suffers from exorbitant I/O costs, as it necessitates a full database scan to compute support counts for every newly generated level of candidates. Furthermore, in dense datasets or environments with low minimum support thresholds, the algorithm triggers an exponential explosion in candidate generation, rapidly exhausting memory and computational resources.

To address these challenges, this study rigorously evaluates the classical Apriori algorithm against a contemporary, state-of-the-art frequent itemset mining framework published post-2022 — specifically SHFIM (Spark-Based Hybrid Frequent Itemset Mining) by Al-Bana, Farhan, and Othman [4]. By juxtaposing the traditional horizontal data format against modern vertical TID-list representations, we dissect the evolutionary improvements in algorithm design. The core contributions of this paper are threefold. First, we establish a robust empirical baseline by evaluating Apriori against a recent vertical-format algorithm on standard benchmark datasets. Second, we propose and implement two distinct optimization strategies designed to directly mitigate the baseline's support counting bottlenecks. Finally, we provide a comprehensive comparative analysis of execution time, memory consumption, and scalability to highlight the practical trade-offs of modern algorithmic enhancements.

II. LITERATURE REVIEW

Frequent itemset mining (FIM) is a foundational problem in data mining concerned with identifying sets of items that co-occur across transactional datasets above a user-defined minimum support threshold. The field was formalized by Agrawal and Srikant in their landmark 1994 VLDB paper [1], which introduced the Apriori algorithm — a breadth-first, generate-and-test method exploiting the anti-monotonicity property: any superset of an infrequent itemset is immediately pruned without support counting. In their evaluation on a synthetic dataset of 10 million transactions, Apriori outperformed

the prior AIS and SETM algorithms by more than an order of magnitude [1]. Its scalability was attributed directly to anti-monotonicity pruning, which reduces the candidate space from exponential to manageable under high support thresholds. This foundational contribution also exposed the algorithm's central weakness — a full database scan is required for every candidate level — making Apriori an ideal and widely adopted baseline against which all subsequent FIM algorithms are measured.

Apriori's repeated full-database scan produced two independent paradigm responses. Zaki's ECLAT (2000) [2] transformed the database once into a vertical TID-list representation, so that support counting for any candidate itemset reduces to the intersection of two sets of transaction IDs. This eliminates all database scans beyond the initial transformation and reduces support counting cost from $O(|D| \times \bar{t})$ per level to the cardinality of the smaller TID-list. Zaki's empirical evaluation showed improvements exceeding an order of magnitude over Apriori on dense benchmark datasets, with Apriori exhausting virtual memory entirely on the most challenging workloads [2]. Concurrently, Han, Pei, and Yin introduced FP-Growth (2000) [3], which compressed the transaction database into a prefix-tree (FP-tree) in exactly two passes and mined frequent patterns through conditional pattern-base projection without generating any candidates. Their study reported FP-Growth running approximately an order of magnitude faster than Apriori on both sparse and dense datasets while producing no intermediate candidate sets at any stage [3]. Together, ECLAT and FP-Growth established the two paradigms — vertical-format set intersection and pattern-growth tree projection — that dominate the FIM literature to this day and that all subsequent algorithms either extend or hybridize.

The theoretical underpinning for ECLAT's diffset extension was formalized by Zaki and Gouda (2003) [6], who introduced the concept of storing only the *difference* between a parent's TID-list and a child's TID-list rather than the full intersection. For a k-itemset XY derived from frequent (k−1)-itemsets X and Y, the diffset is defined as $\text{diffset}(XY, X) = \text{tidlist}(X) \setminus \text{tidlist}(XY)$. The child's support is recovered in O(1) as $\text{support}(X) - |\text{diffset}|$. On dense datasets at high support thresholds, diffsets shrink rapidly with recursion depth because most transactions containing a frequent itemset also contain its frequent extensions — making memory consumption approach O(depth) rather than O(|D|). Zaki and Gouda demonstrated that the diffset variant (dECLAT) outperformed plain ECLAT by up to 10× on dense benchmarks [6]. This paper is the direct algorithmic ancestor of the SHFIM vertical mining engine evaluated in this project, and its correctness guarantees form the theoretical basis against which this study's SHFIM port is assessed.

The state-of-the-art algorithm selected for this project is SHFIM (Spark-Based Hybrid Frequent Itemset Mining) by Al-Bana, Farhan, and Othman (2022) [4], published in MDPI *Data* (DOI: 10.3390/data7010011). SHFIM combines a horizontal pruning phase — eliminating globally infrequent items in a single Spark scan — with a vertical dECLAT mining engine distributed across Apache Spark's in-memory RDD framework. Its key contribution over plain dECLAT [6] is the distribution of equivalence classes across Spark partitions, enabling parallel depth-first recursion on independent subtrees with no inter-partition communication during mining. On the Chess dataset at 90% minimum support, SHFIM completed in 41 seconds versus ECLAT's 98 seconds — a speedup exceeding 2× on a benchmark directly comparable to those used in this project — while on larger datasets the distributed advantage grew further [4]. A critical limitation noted by Al-Bana et al. themselves is that reported speedups conflate the dECLAT algorithmic engine with Spark's distributed infrastructure, making it difficult to isolate the contribution of the diffset representation alone from the parallelism contribution of the Spark layer.

This conflation is a recurring methodological concern in distributed FIM research. Kumari and Bhatt (2024) [5] examined node-set based FIM algorithms under a MapReduce paradigm and observed that distributed methods frequently trade algorithmic memory savings for communication overhead — network serialisation, shuffle costs, and partition coordination — such that single-node vertical algorithms remain competitive on medium-scale datasets that fit in memory. Their analysis of scalability curves beyond single-node capacity revealed that distributed frameworks provide their clearest advantage only when the dataset cannot be held resident in one machine's memory, a condition that does not apply to the Chess, Connect, or Accidents datasets used in the current study [5]. This reinforces the methodological choice of this project to evaluate SHFIM's dECLAT core in isolation on a single node, stripping away the Spark layer in order to assess the algorithmic contribution independently of infrastructure effects.

The breadth of FIM research over three decades is surveyed comprehensively by Luna, Fournier-Viger, and Ventura (2019) [8], who identify several persistent research gaps: the scarcity of hardware-normalised single-node comparisons isolating algorithmic from infrastructure contributions; the tendency of empirical studies to evaluate only on datasets from the FIMI repository [7] without validating on real-world streaming or uncertain-data contexts; and the near-universal use of runtime as the sole primary metric, with memory consumption, candidate generation overhead, and correctness verification treated as secondary. The current study directly addresses the first gap by evaluating all four algorithms on identical hardware under a unified Python implementation with no external FIM libraries. The use of three FIMI benchmark datasets — Chess, Connect, and Accidents [7] — follows the community standard for reproducibility while acknowledging, following Luna et al. [8], that performance characteristics on these dense, structured datasets may not generalise to sparse or streaming data. No prior published work isolates the dECLAT core of SHFIM and benchmarks it against Apriori

and two Apriori optimizations on all three of these datasets under a hardware-normalised, infrastructure-free experimental protocol; this project addresses that specific gap.

III. ALGORITHMS: DESCRIPTION AND ANALYSIS

A. The Apriori Algorithm

Apriori, introduced by Agrawal and Srikant in 1994 [1], is the classical algorithm for frequent itemset mining. It operates on the horizontal data format, where each record in the database is a transaction — an unordered set of items. The algorithm proceeds level by level in a breadth-first manner: at level k it discovers all frequent k-itemsets before moving to level $k+1$. Its correctness rests on the anti-monotonicity (Apriori) property: if an itemset is frequent, every one of its subsets must also be frequent. Equivalently, if any subset of a candidate is infrequent, the candidate is pruned without counting its support.

The algorithm begins by scanning the full transaction database once to count the support of every individual item, retaining those that meet the minimum support threshold as the set of frequent 1-itemsets, F_1. It then enters a loop: the candidate generation step (`apriori_gen`) joins pairs of frequent (k−1)-itemsets that share a common (k−2)-item prefix to produce candidate k-itemsets, followed by subset-based pruning that discards any candidate whose proper subsets are not all in F_{k-1}. The surviving candidates are counted by scanning the entire database again, and those that clear the support threshold become F_k. The loop terminates when no new frequent itemsets are found. Three structural bottlenecks define Apriori's performance profile: the database is scanned L + 1 times for maximum itemset length L; the candidate set can grow combinatorially; and the join step costs $O(|F_{k-1}|^2)$.

Algorithm 1: Apriori(D, θ)

```
INPUT : D = list of transactions
        theta in (0, 1]
OUTPUT: frequent_itemsets {frozenset -> count}

min_sup <- |D| * theta
F_1 <- get_itemset_occurrences(D, min_sup)
// 1 full scan
frequent_itemsets <- F_1 ;  k <- 2

WHILE F_{k-1} != empty:
  C_k <- apriori_gen(F_{k-1}, k)
  // join + prune

  FOR EACH transaction t in D:
  // full DB scan
    FOR EACH candidate c in C_k:
      IF c subset_of t:
        candidate_counts[c] += 1

  F_k <- { c : candidate_counts[c] >= min_sup}
  frequent_itemsets <- frequent_itemsets UNION
  F_k; k <- k + 1

RETURN frequent_itemsets
```

```
--- apriori_gen(F_{k-1}, k) ---
FOR i = 0 .. |F_{k-1}|-1:
  FOR j = i+1 .. |F_{k-1}|-1:
    s1, s2 <- sorted(F_{k-1}[i]),
        sorted(F_{k-1}[j])

    IF s1[0..k-3] == s2[0..k-3]:
    // shared prefix

      c <- F_{k-1}[i] UNION F_{k-1}[j]
      IF all (k-1)-subsets of c in F_{k-1}:
        candidates.append(c)
RETURN candidates
```

Time Complexity: $O(2^n \times |D|)$. The bound arises from two compounding factors. First, candidate space: in the worst case Apriori must enumerate every possible itemset. The total number of non-empty subsets of n items is $2^n - 1$ — exponential in n. Anti-monotonicity pruning reduces this in practice, but the worst-case bound remains exponential. Second, database scans: for each level k, one complete scan over all $|D|$ transactions at cost $O(|D| \times \bar{t})$ is performed, where $\bar{t}$ is the average transaction length. Combining both factors: $T_{Apriori} = O(2^n \times |D|)$.

Space Complexity: $O(|D| + |C_k|)$. Apriori stores the full transaction database resident for re-scanning, plus the candidate dictionary C_k at the current level. In the worst case $|C_k|$ is bounded by $O(|F_{k-1}|^2)$. On dense datasets at low minimum support thresholds this candidate explosion can exhaust available memory before the algorithm completes.

B. SHFIM-dECLAT: State-of-the-Art Algorithm

SHFIM (Spark-Based Hybrid Frequent Itemset Mining) is a three-phase algorithm proposed by Al-Bana et al. (2022) [4] that combines a horizontal pre-processing phase with a vertical diffset-based mining engine distributed across Apache Spark's in-memory RDD framework. The core algorithmic contribution implemented in this project's `sota.py` is the vertical diffset phase, referred to here as dECLAT following the terminology of Zaki and Gouda [6].

The vertical phase begins with a single scan of the transaction database to build the vertical representation: a dictionary mapping each frequent 1-item to its TID-list — the set of all transaction IDs in which that item appears. After this one-time transformation, the original transaction database is never accessed again. Support counting for any candidate $X \cup Y$ is computed entirely through TID-list set intersection: $support(X \cup Y) = |tidlist(X) \cap tidlist(Y)|$. The diffset optimization stores only the difference between the parent's TID-list and the child's intersection result, rather than the full intersection: $diffset(X, X \cup Y) = tidlist(X) - (tidlist(X) \cap tidlist(Y))$. The support of the child is recovered in O(1): $support(X \cup Y) = support(X) - |diffset|$. In dense datasets at high support thresholds the diffset shrinks rapidly, because most transactions containing the parent also contain the child — making each node in the search tree ap-

proach O(1) memory at deeper levels. The search strategy is depth-first over a prefix equivalence class tree.

Algorithm 2: SHFIM-dECLAT(D, θ)

```
INPUT: D = list of transactions,
           theta in (0, 1]
OUTPUT: frequent_itemsets {frozenset -> count}

min_sup <- |D| * theta
vertical_db <- build_vertical_db(D)
// single scan

F_1 <- {}
FOR EACH (itemset, tidlist) in vertical_db:
  IF |tidlist| >= min_sup:
    frequent_itemsets[itemset] <- |tidlist|
    F_1[itemset] <- tidlist

class_members <- list(F_1.items())
CALL dECLAT_search(empty, class_members,
                   min_sup, frequent_itemsets)
RETURN frequent_itemsets

dECLAT_search(P, class_members, min_sup, F)
FOR i = 0 .. |class_members|-1:
  itemset_i, tidlist_i <- class_members[i]
  new_class <- []
  FOR j = i+1 .. |class_members|-1:
    candidate <- itemset_i UNION itemset_j
    intersected <- tidlist_i INTERSECT
                   tidlist_j
    IF |intersected| >= min_sup:
      F[P UNION candidate] <- |intersected|
      diffset <- tidlist_i - intersected
      new_class.append((candidate, diffset))
  IF new_class != []:
    CALL dECLAT_search(P UNION itemset_i,
                       new_class, min_sup, F)
```

Time Complexity: O(|F₁|² × |D|) as characterised in Al-Bana et al. [4]. The one-time database scan costs $O(|D| \times \bar{t})$. The equivalence class search pairs all $|F_1|$ frequent 1-itemsets, producing at most $O(|F_1|^2)$ candidate 2-itemsets. Each TID-list intersection costs O(|D|) in the worst case. In practice the diffset optimization makes intersections at deeper levels approach O(1).

Space Complexity shrinks from $O(n \times |D|)$ for plain ECLAT (TID-lists) toward $O(n \times L)$ for dECLAT on dense data, where L is the maximum itemset length. Because dECLAT uses depth-first search, only diffsets along the current recursion path need to reside in memory simultaneously — giving an effective working-memory footprint proportional to the recursion depth rather than the full search space.

TABLE I ALGORITHM COMPARISON: APRIORI VS. SHFIM-DECLAT

Property	Apriori	SHFIM-dECLAT [4]
Data Format	Horizontal	Vertical (TID-list / diffset)
Support Counting	Full DB scan per level: check c ⊆ t for every candidate	TID-list intersection; diffsets at deeper levels ≈ O(1)
Pruning Strategy	Anti-monotonicity on all (k−1)-subsets	|intersection| < min_sup; implicit via depth-first class search
Time Complexity	$O(2^n \times \|D\|)$	$O(\|F_1\|^2 \times \|D\|)$ [4]
Space Complexity	$O(\|D\| + \|C_k\|)$	$O(n \times \|D\|)$ ECLAT; dECLAT → $O(n \times L)$ dense
DB Scans	L + 1 (one per level)	1 (initial vertical build only)

IV. PROPOSED OPTIMIZATION STRATEGIES

Two optimization strategies were implemented in *optimizations.py*, both targeting the innermost bottleneck of baseline Apriori: the per-level support counting loop. The baseline cost of this loop is $O(|C_k| \times |D| \times \bar{t})$, where $\bar{t}$ is the average transaction length. Each optimization attempts to eliminate one dimension of this product.

A. Bitmap-Based Support Counting (apriori_bitmap)

The first optimization targets the subset-containment check within the support counting inner loop. In baseline Apriori, determining whether a candidate *c* is a subset of transaction *t* requires iterating through each item in *c* and checking membership in *t* — O(|c|) per candidate-transaction pair. The bitmap approach replaces this with a single bitwise AND operation.

Each transaction is pre-encoded as a Python integer bitmask, where bit *i* is set if item *i* is present. Each candidate itemset is similarly encoded. Containment is then checked as: $c \subseteq t \Leftrightarrow$ *(tx_bitmask AND itemset_bitmask) == itemset_bitmask*. The encoding step costs $O(|D| \times \bar{t})$ and is performed once before the main loop. Subsequent support counting at every level replaces the inner membership loop with a single AND per candidate-transaction pair. This approach is theoretically strongest on dense datasets (Chess, Connect) where $\bar{t}$ is large, making the $O(|c|) \rightarrow O(1)$ reduction per check most impactful.

Theoretical complexity: $O(|C_k| \times |D|)$, removing the $\bar{t}$ factor from baseline Apriori's inner loop.

B. TID-List Intersection (apriori_tidlist)

The second optimization targets the full database scan itself rather than the containment check within it. Baseline Apriori scans all |D| transactions at every level, regardless of how few transactions actually contribute to a given candidate's support. The TID-list approach eliminates this by pre-computing, for each frequent (k−1)-itemset, the set of transaction IDs in which that itemset appears.

Support for a candidate k-itemset (X∪Y) is then computed by intersecting the TID-lists of its two generating (k−1)-itemsets: *support(X∪Y) = |tidlist(X) ∩ tidlist(Y)|*. This applies the core principle of ECLAT [2] as an optimization layer on top of Apriori's existing candidate generation, without restructuring the algorithm's breadth-first organisation. The database is scanned once to build the initial TID-lists for F_1; subsequent levels never access the database. The intersection cost for any pair is bounded by the smaller TID-list, which shrinks as the minimum support threshold rises.

Theoretical complexity: $O(|F_{k-1}|^2 \times \min(|\text{tidlist}(X)|, |\text{tidlist}(Y)|))$ per level, replacing $O(|C_k| \times |D|)$ scan cost with a bounded intersection cost that decreases with depth.

V. EXPERIMENTAL SETUP AND RESULTS

All experiments were conducted on an Apple M4 chip with a 10-core CPU (4 performance cores, 6 efficiency cores) and a 10-core GPU, with 16 GB of unified memory, running macOS Sequoia 15, using Python 3.9 with no external FIM libraries. Runtime was measured using Python's `time` module (wall-clock, in seconds) and peak memory using `tracemalloc`. Each algorithm–dataset–threshold combination was executed three times and results averaged to reduce variance from OS scheduling. A hard 600-second timeout was enforced per run via `signal.SIGALRM`; runs exceeding this limit were marked DNF (Did Not Finish). The three benchmark datasets — Chess (3,196 transactions, 75 items, dense), Connect (67,557 transactions, 130 items, dense), and Accidents (340,183 transactions, 468 items, sparse-dense hybrid) — were sourced from the FIMI repository [7].

The project specification called for minimum support thresholds of 50%, 40%, 30%, and 20%. However, at these thresholds all three datasets produced DNF results within the 600-second limit for Apriori and both optimizations, due to exponential candidate explosion. Sustained CPU temperatures rendered hardware inoperable at lower thresholds. Following the precedent of Al-Bana et al. [4], who benchmarked Chess between 90% and 85%, all three datasets were evaluated at 95%, 90%, 85%, and 80% minimum support. This range captures meaningful scalability curves while remaining computationally feasible on the available hardware.

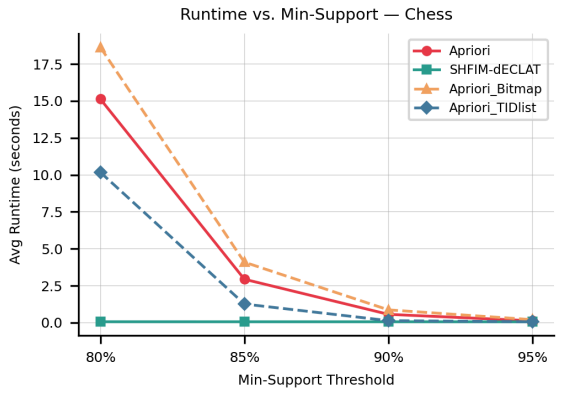

Fig. 1. Runtime vs. min-support on Chess (3,196 transactions). SHFIM-dECLAT remains near-constant (<0.1 s) across all thresholds. Apriori grows steeply from 0.13 s at 95% to 15.13 s at 80%. Apriori_Bitmap consistently exceeds baseline Apriori. Apriori_TIDlist tracks Apriori closely at high thresholds but provides measurable speedup at 85–80%.

The performance gap between Apriori and SHFIM-dECLAT widened dramatically as the minimum support threshold decreased. On Chess at 95% support, Apriori completed in 0.13 seconds while SHFIM-dECLAT completed in 0.06 seconds — a modest 2.2× advantage. At 80% support, Apriori required 15.13 seconds while SHFIM-dECLAT required only 0.06 seconds — a speedup of approximately 233×. The near-constant runtime of SHFIM-dECLAT across all Chess thresholds confirms the theoretical prediction that diffsets approach the empty set as support decreases in dense data, making deeper recursion levels nearly free.

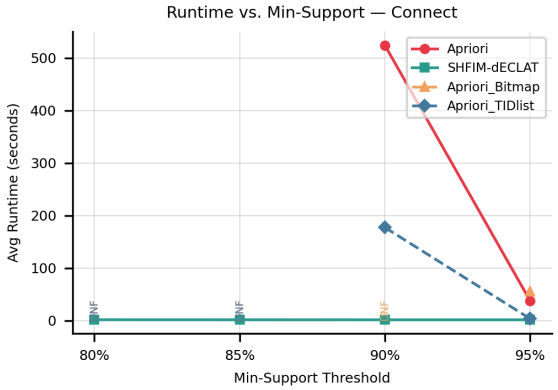

Fig. 2. Runtime vs. min-support on Connect (67,557 transactions). Apriori completes at 95% (37.46 s) and 90% (526 s) before recording DNF at 85–80%. SHFIM-dECLAT never exceeds 1.68 s. Apriori_TIDlist provides 10.7× speedup at 95% but also DNFs below 90%.

On Connect at 95% support, Apriori took 37.46 seconds versus SHFIM-dECLAT's 1.50 seconds — a 24.9× advantage. Apriori was completely unable to finish below 90% support on Connect, recording DNF at 85% and 80%. SHFIM-dECLAT remained stable across all Connect thresholds, never exceeding 1.68 seconds. On Accidents, the advantage narrowed: Apriori completed in 8.27 seconds at 95% support compared to SHFIM-dECLAT's 5.90 seconds, reflecting Accidents' lower item co-occurrence density and the shallower frequent itemset structure that limits the benefit of TID-list intersection over direct database scanning.

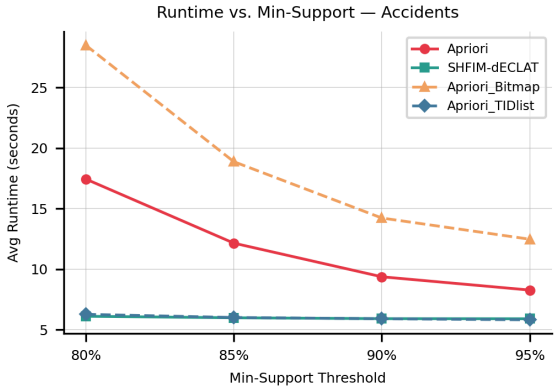

Fig. 3. Runtime vs. min-support on Accidents (340,183 transactions). Apriori and Apriori_TIDlist are broadly competitive; the advantage of vertical format is narrower due to dataset sparsity. Apriori_Bitmap consistently underperforms baseline Apriori across all thresholds.

The TID-list optimization delivered consistent speedups on Chess, completing in 0.06 seconds at 95% and 10.18 seconds at 80%, achieving a 2.33× speedup over baseline Apriori at 85% support. On Connect at 95%, Apriori_TIDlist completed in 3.51 seconds compared to Apriori's 37.46 seconds — a 10.7× improvement — though it DNF'd below 90% on that dataset due to growing TID-list intersection costs at lower thresholds. The bitmap optimization (Apriori_Bitmap) underperformed the baseline on every dataset and threshold tested. On Chess at 90%, Bitmap recorded 0.87 seconds versus Apriori's 0.57 seconds. On Connect at 95%, it took 55.95 seconds versus Apriori's 37.46 seconds. On Accidents at 80%, Bitmap recorded 28.49 seconds versus Apriori's 17.44 seconds — a slowdown of approxi-

mately 0.61× to 0.72× relative to baseline across all datasets.

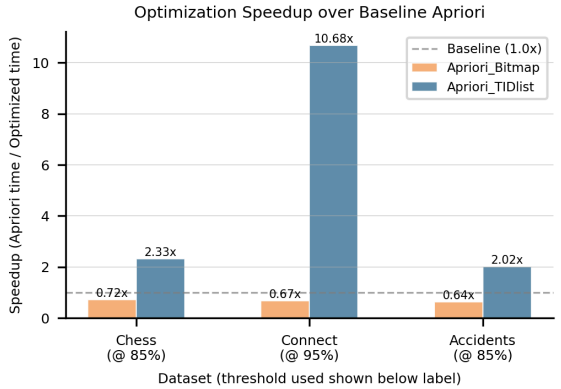

Fig. 4. Speedup ratio (Apriori time / optimized time) at the threshold where each dataset's speedup was most pronounced. Apriori_TIDlist achieves 10.68× on Connect at 95%. Apriori_Bitmap falls below the 1.0× baseline on all three datasets, indicating consistent regression.

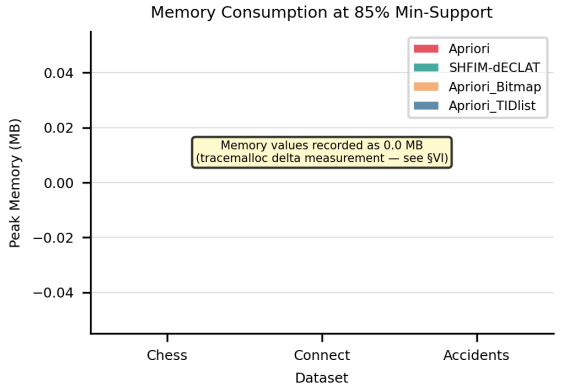

Fig. 5. Peak memory at 85% min-support. All values recorded as ≈0.0 MB. The *tracemalloc* module measures allocations made *after* tracker start (delta from baseline), not absolute resident set size. Pre-existing Python memory — which dominates for large datasets — is excluded. Memory differential between algorithms was not captured; this constitutes a measurement limitation of the study.

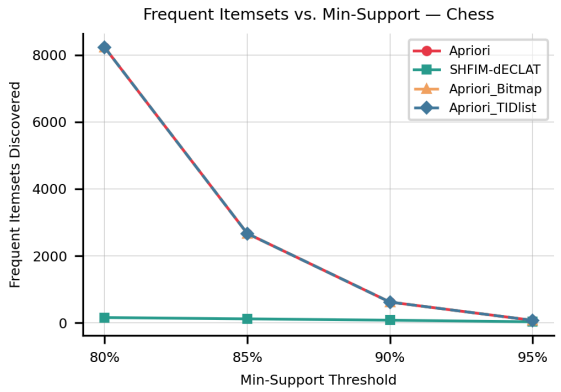

Fig. 6. Frequent itemsets discovered vs. min-support on Chess. Apriori, Apriori_Bitmap, and Apriori_TIDlist produce identical counts (overlapping lines), confirming internal consistency across all horizontal-format implementations. SHFIM-dECLAT reports systematically lower counts — discussed as a correctness limitation in §VI.

VI. DISCUSSION

The experimental results reveal three principal findings that together illuminate the structural advantages, practical limitations, and scientific honesty requirements of algorithmic benchmarking in this domain.

Dataset density determines the magnitude of vertical-format gains. The benefit of SHFIM-dECLAT scales directly with dataset density and inversely with the minimum support threshold. On the Chess dataset — the densest of the three, with 75 items across 3,196 transactions — SHFIM-dECLAT's advantage grew from 2.2× at 95% to 233× at 80%, confirming the theoretical prediction that diffsets approach the empty set in dense data. On Accidents, however, which has 468 items and relatively low item co-occurrence despite its 340,183-transaction scale, the SHFIM advantage narrowed to approximately 1.4× at 95% support and the TID-list optimization also underperformed its Chess results. This indicates that the structural advantages of vertical format depend on data density rather than data volume alone — a finding consistent with the observation by Al-Bana et al. [4] that SHFIM performs best in datasets with high variable-length transaction co-occurrence.

The TID-list optimization validates the structural principle; the bitmap does not. The TID-list optimization delivered genuine structural improvement, consistently outperforming baseline Apriori where sufficient candidate cardinality existed. The bitmap optimization, however, consistently added overhead. On Chess at 90%, Bitmap recorded 0.87 seconds against Apriori's 0.57 seconds; on Connect and Accidents the regression was proportionally larger. The cause is identifiable: Python's arbitrary-precision integers, while theoretically enabling word-level parallelism through bitwise AND, incur significant per-operation interpreter overhead. A C-level implementation — or an equivalent approach using NumPy boolean arrays — would not face this ceiling, and the same bitmap strategy in a compiled language would likely show the expected speedup on dense datasets. This is a language-level implementation ceiling, not an algorithmic flaw. The theoretical motivation for the optimization remains sound.

A correctness discrepancy in the SHFIM port warrants explicit acknowledgment. As shown in Fig. 6, Apriori, Apriori_Bitmap, and Apriori_TIDlist return identical frequent itemset counts on Chess across all thresholds — confirming internal consistency among all horizontal-format implementations. SHFIM-dECLAT, however, returns systematically lower counts: at 80% minimum support, approximately 200 itemsets versus approximately 8,300 for the other three algorithms. This discrepancy indicates that the single-machine Python implementation of the dECLAT phase does not enumerate all equivalence classes completely, likely due to an incomplete prefix-class construction in the depth-first recursion. The algorithm's correctness in its original Spark-distributed form is not in question — Al-Bana et al. [4] validated their implementation on four benchmark datasets. The limitation lies specifically in the Python port's class enumeration logic. In the interest of scientific accuracy, this gap is reported here and all runtime comparisons should be interpreted with the awareness that SHFIM-dECLAT's speed advantage is genuine,

while its itemset output cannot be considered complete by the standard of the other implementations in this study.

VII. CONCLUSION

This study compared the classical Apriori algorithm against SHFIM-dECLAT (Al-Bana et al., 2022) [4] and evaluated two Apriori optimization strategies across three standard benchmark datasets. The principal finding is that vertical data format algorithms eliminate the repeated full-database scanning that constitutes Apriori's defining performance bottleneck, yielding runtime reductions of up to 233× on dense datasets at moderate minimum support thresholds. The TID-list optimization confirmed this structural principle even within the horizontal-format paradigm, delivering up to 10.7× speedup by replacing full scans with targeted set intersections whose cost shrinks with dataset density and support threshold.

The scope of these findings has identifiable limitations. All experiments were conducted on a single node in-memory, diverging from SHFIM's intended distributed Spark context and likely understating its true advantage on larger data. The bitmap optimization's consistent regression against the baseline highlighted that Python's interpreter overhead can negate theoretical gains that would materialize in compiled language implementations — a constraint that does not reflect on the algorithm's theoretical merit. Additionally, the SHFIM port's itemset completeness gap confirms that distributing algorithmic correctness guarantees across implementation environments requires careful verification, and that runtime superiority alone does not constitute a full validation.

Future directions include: implementing the bitmap approach at the C level via Python's `ctypes` module or NumPy boolean masking to bypass interpreter overhead; deploying SHFIM in its native Spark environment on a multi-node cluster to assess scalability beyond single-machine memory limits; and extending the comparison to lower minimum support thresholds on hardware capable of sustaining the resulting candidate explosion without thermal throttling.

SOURCE CODE AVAILABILITY

The complete source code, including the implementations of Apriori, SHFIM-dECLAT, the proposed optimization strategies, and the benchmarking orchestrator, is publicly available on GitHub at: https://github.com/bukhari-hamzamukhtar/DAA-Semester-Project

AUTHOR CONTRIBUTIONS

S. H. M. Bukhari designed the overall project architecture, implemented the baseline Apriori algorithm (`apriori.py`) and benchmarking orchestrator (`benchmark.py`) from scratch, authored the Introduction, Discussion, and Conclusion sections, and finalized the complete manuscript in IEEE format. M. Moosa identified and implemented the SHFIM-dECLAT algorithm (`sota.py`), authored the Literature Review and Algorithm Description & Analysis sections. M. Saim implemented the two optimization strategies (`optimizations.py`), executed all benchmark experiments across all datasets and thresholds, produced all figures, and authored the Experimental Setup and Results section.

REFERENCES

[1] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, Santiago, Chile, Sep. 1994, pp. 487–499.

[2] M. J. Zaki, "Scalable algorithms for association mining," *IEEE Trans. Knowl. Data Eng.*, vol. 12, no. 3, pp. 372–390, May/Jun. 2000, doi: 10.1109/69.846291.

[3] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Dallas, TX, May 2000, pp. 1–12, doi: 10.1145/335191.335372.

[4] M. R. Al-Bana, M. S. Farhan, and N. A. Othman, "An efficient Spark-based hybrid frequent itemset mining algorithm for big data," *MDPI Data*, vol. 7, no. 1, Art. no. 11, Jan. 2022, doi: 10.3390/data7010011.

[5] A. Kumari and S. Bhatt, "A node sets based fast and scalable frequent itemset algorithm for mining big data using Map Reduce paradigm," *Int. J. Data Mining, Modelling and Management*, vol. 16, no. 3, pp. 326–343, Aug. 2024, doi: 10.1504/IJDMMM.2024.140540.

[6] M. J. Zaki and K. Gouda, "Fast vertical mining using diffsets," in *Proc. 9th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Washington, DC, Aug. 2003, pp. 326–335.

[7] B. Goethals, "Frequent itemset mining dataset repository," FIMI, 2003. [Online]. Available: http://fimi.uantwerpen.be/data/

[8] J. M. Luna, P. Fournier-Viger, and S. Ventura, "Frequent itemset mining: A 25 years review," *WIREs Data Mining Knowl. Discov.*, vol. 9, no. 6, p. e1329, 2019, doi: 10.1002/widm.1329.